

A 90-MINUTE LECTURE ON ARCHITECTURE, ECONOMICS, AND AI LABOR

The Hidden Cost of Delegation

The Economics of Complexity Ownership

Delegation is an architectural and economic decision about where complexity will live — and who will own it.

Libraries · Frameworks · Platforms · AI agents

The arc

0–10 min

The wrong question

Build vs buy hides the real bill.

10–25 min

Complexity relocates

Less code is not less complexity.

25–42 min

Mental models & tax

Elegance, gravity, four costs.

42–55 min

Who should own it

A default for code-owned systems.

55–78 min

AI changes the equation

Labor vs ownership vs acceptance.

78–90 min

A decision framework

Questions, takeaways, discussion.

Don't reinvent the wheel.

Usually correct. Incomplete as an economic test.

We measure the work that disappears

- ▶ A tool can delete thousands of lines and install a mental model.
- ▶ A framework can clean the app and capture how the team thinks.
- ▶ A programming model can make concurrency elegant — and global.
- ▶ An agent can ship a feature in minutes and create a verification problem.

Delegation is an architectural and economic decision about where complexity will live and who will own it.

Once this is the starting point, unrelated choices look like the same problem.

Part I · ≈ 10 minutes

The wrong question

Build versus buy, as usually presented, is a false comparison.

How the comparison is sold

Build

- 2 weeks

Use the tool

- 2 days
- The tool wins.

The real bill

Build

- implementation
- testing
- maintenance

Use the tool

- learning · integration · configuration
- operational knowledge · debugging
- upgrades · limitations · workarounds
- dependency management · migration

Not: how much code does this save?

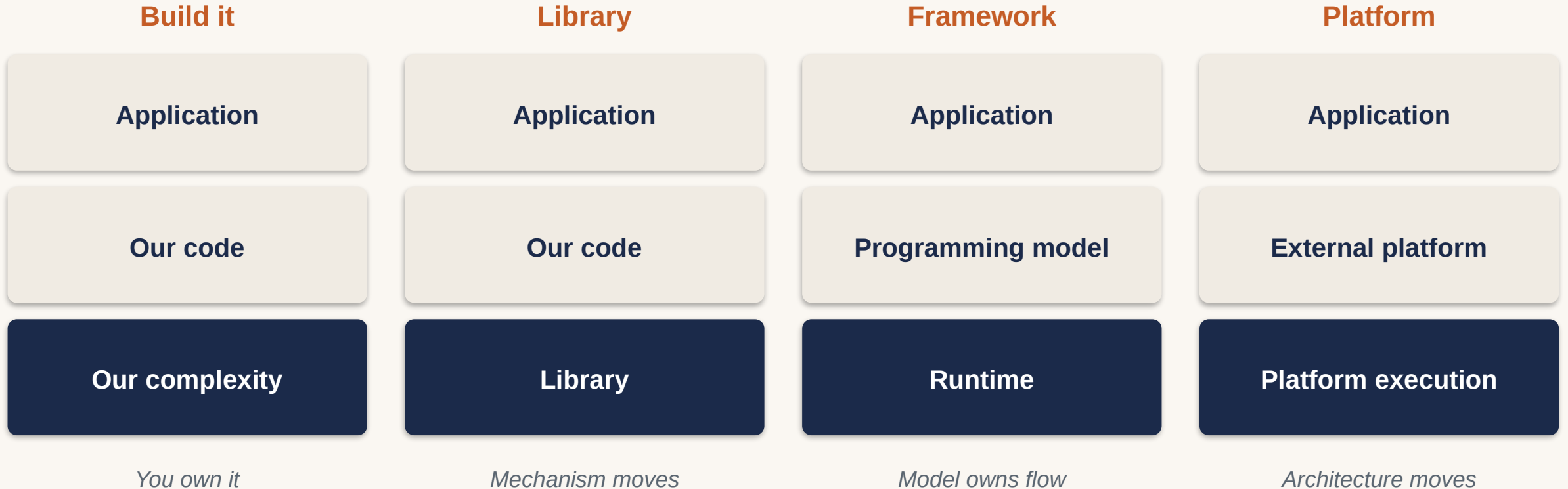
Ask: what is the total engineering cost of owning this decision?

Part II · ≈ 15 minutes

Complexity relocates

Less code and less complexity are not the same thing.

It is relocated



Less code is not less complexity.

The reduction in application code is real. So is the increase in dependency.

Using a capability vs living in a model

A library can leave you in charge

- how components interact
- where policy lives
- how state is managed
- what success and failure mean
- how pieces are composed

A platform can own the execution

- topology and scheduling
- queues, retries, state
- config and deploy model
- operational behavior
- the application is expressed inside it

Open source does not mean you own the architecture

- ▶ You can read the source. You can even fork it.
- ▶ If you depend on its runtime, config model, workflow semantics, state model, and ops assumptions — you are coupled.
- ▶ Source availability and architectural ownership are different properties.
- ▶ Access to their code is not control over how your application is structured around it.

Part III · ≈ 17 minutes

Mental models and the architecture tax

The least visible cost is how the team is forced to think.

How much of the codebase must think in this model?

Shallow

- A utility library.
- Learn the API.
- Keep thinking in the application's concepts.

Deep

- Effect systems, actors, reactive, workflows, streams.
- Powerful — and demanding.
- The conceptual model becomes part of the application.
- A local choice becomes a global commitment.

Local abstraction vs global programming model

Local

- domain
- application logic
- infrastructure
- library A / B / C

Global

- programming model
- component A
- component B
- almost everything else

Local elegance can hide a global commitment.

Not: which code looks cleaner? Ask: does the reduction in complexity justify how much of the system must adopt the model?

The dark side of “the right tool”

- ▶ There was a tool. It promised to eliminate implementation.
- ▶ It looked like the responsible choice.
- ▶ Then: learn the model, learn the boundaries, pay for workarounds.
- ▶ The late question: why didn't I just write the thing?
- ▶ Usually the tool was fine. The problem was more specific than the abstraction. You paid for generality. Adoption itself has a cost.

The architecture tax

What you needed

A → **B**

A specific path. A bounded problem. Not a platform.

What the platform may install

- routing
- transformation
- scheduling
- state · retries · backpressure
- parallelism · observability
- operational assumptions

Four costs of a technical choice

1. Implementation

How hard is it to build ourselves?

2. Adoption

Understand, integrate, learn.

3. Ownership

Operate, debug, maintain, upgrade, govern.

4. Exit

How hard is it to remove later?

Architectural reversibility is itself a form of value.

A cheap decision that is easy to reverse is not the same as a cheap decision that locks the model for years.

Technology gravity

1

A technology

2

Extra components

3

New ops practices

4

New deploy model

5

New team knowledge

6

A new architecture

Part IV · ≈ 13 minutes

Who should own this complexity?

The goal is appropriate ownership — not maximum ownership.

Possible owners

Application

Owns the important behavior.

Library

Owns a mechanism.

Framework

Owns part of the lifecycle.

Platform

Owns execution.

External service

Owns an entire capability.

The test

Can we understand, test, debug, and change the behavior that matters?

How much architectural control am I giving away — and is the reduction in complexity worth it?

Not: frameworks good, platforms bad. A code-centric framework still owns structure, lifecycle, DI, execution, tests, extension.

Code owns the architecture. Libraries provide capabilities. Frameworks must justify their programming model. External execution platforms are exceptions, not defaults.

A bias toward architectural ownership — not a ban on tools.

Default path for code-owned projects

1

Can a library solve the mechanism?

2

Can a framework help without taking the architecture?

3

Can a thin custom layer express the specific behavior?

4

Only then consider a platform.

Not “build it because I can.” Not “use it because someone already built it.”

Ask: which ownership boundary gives the lowest total engineering cost?

Part V · ≈ 23 minutes

AI agents change the equation

Traditional tools delegate mechanisms or execution. Agents can delegate labor itself.

Not the far end of library → framework → platform

Conventional tool

- Human decides.
- Tool executes a procedure we designed.

Coding agent

- Receives goal, constraints, acceptance.
- Researches, plans, implements, tests, debugs, refactors.
- May choose what to inspect and how to revise.
- Participates in designing the procedure.

Labor is not ownership

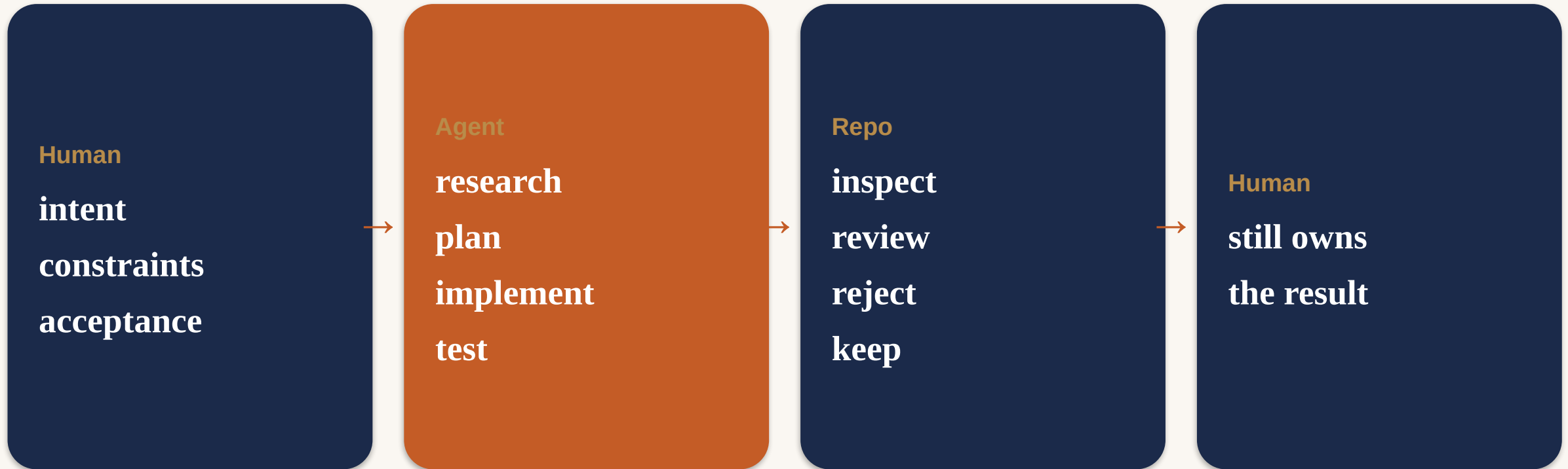
Labor

- Work performed to produce an outcome.

Ownership

- What should be done.
- Under which constraints.
- What counts as acceptable.
- Whether to trust and adopt the result.

Agents can produce owned artifacts



The important property is not that AI produced it. It is that the output can become an owned artifact of the codebase.

Open-source platform vs agent-written code

Adopt their platform

- Your system
- → their runtime
- → their architecture
- You can read the source. You still run their model.

Agent implements your problem

- Your problem
- → agent labor
- → tailored implementation
- → your repository
- Large labor, no new owner.

The criterion is verifiability, not difficulty

Highly delegatable

- rename references
- migrate this API
- generate tests for these cases
- implement a known algorithm
- convert a representation

Judgment-heavy

- choose the architecture
- interpret an ambiguous requirement
- decide a business trade-off
- determine acceptable risk

Two very different loops

You own the problem

- You define the problem.
- AI solves it.

AI owns the interpretation

- AI interprets the problem.
- AI defines what good means.
- AI solves its interpretation.

Collaboration (grilling) sits in between

- ▶ Propose an approach.
- ▶ What assumptions are you making?
- ▶ What could make this design wrong?
- ▶ Give the strongest argument against your own approach.
- ▶ What alternatives did you reject?
- ▶ Exploratory labor stays with the agent. Judgment stays with the human. Especially valuable before implementation.

Self-critique is not independent verification.

An agent can wrap a flawed assumption in a consistent argument — including a critique that preserves the same misconception.

Move humans to judgment, not to every mechanical check

- ▶ Requirement comparison, consistency, bugs, edges, security, architecture, behavior.
- ▶ Much of that labor can be delegated: AI reviewer, security reviewer, tests, static analysis, simulation.
- ▶ The goal is not to eliminate human review.
- ▶ The labor of review can be delegated without delegating ownership of acceptance.

Keep humans responsible for the decisions that matter — not necessarily for all the work.

Stop reading every line if you must. Do not stop owning 'is this acceptable for this system?'

Who owns the decision?

Human does everything

AI assists

AI performs defined labor

AI plans the labor

AI chooses the procedure

AI makes consequential decisions

Further down this path:

- stronger governance
- stronger verification
- more reversibility
- more explicit constraints

Complexity moved to verification

Then

3 days coding

Production was the bottleneck.

Now

2 hours to generate

+ 2 days to understand

The bottleneck moved. It did not vanish.

Automate verification as aggressively as you automate production.

Delegate generation aggressively; delegate acceptance conservatively.

The more autonomous generation becomes, the stronger the evidence for acceptance must be.

Part VI · ≈ 12 minutes

A decision framework

Every engineering decision is partly about the location, ownership, and movement of complexity.

Seven questions, not a feature list

Complexity

Where does it live?

Ownership

Who is responsible for it?

Delegation

Who performs the labor?

Mental model

What must the team think in?

Reversibility

How expensive is exit?

Verification

How do we know it is correct?

Gravity

What else does this attract?

Ask these, specifically

Remove?

What complexity does it remove? “Less code” is not an answer.

Introduce?

Learning, ops, config, debugging, organizational knowledge.

Who owns behavior?

Application, library, framework, platform, service?

How deep is the model?

Local dependency, or must the whole system think in it?

Exit cost?

Replace an implementation — or replace the architecture?

Verification?

How do we know delegated behavior is correct?

Gravity?

What ecosystem does this make likely?

Owned artifact?

Especially with AI: does the result remain ours?

Forms of delegation

- ▶ A platform moves execution complexity outside the application.
- ▶ A framework moves lifecycle complexity into a programming model.
- ▶ A functional runtime moves concurrency into another computational model.
- ▶ A managed service moves operations to a provider.
- ▶ An AI agent moves implementation labor to an autonomous system.
- ▶ None are inherently good or bad. Each rearranges labor, complexity, knowledge, control, ownership, and verification.

Delegate labor when possible. Delegate ownership when the economics genuinely justify it.

And when you delegate ownership, do it deliberately — not accidentally.

The goal is not maximum control.

Owning a hard distributed system yourself can cost more than a mature platform. A demanding programming model can be worth it when the problem requires it.

Takeaways

1

Complexity is relocated, not deleted.

2

Ask total cost of ownership, not lines of code saved.

3

Prefer delegating labor over delegating ownership.

4

Delegate generation aggressively; delegate acceptance conservatively.

5

Find the cheapest sustainable place for complexity — with the control the system actually needs.

Where should this complexity live?

Who should perform the labor, who should make the decision, and who should own the result?

That is the hidden cost of delegation.

Questions · disagreements · live decisions